

Chapter 1

Energy Losses Calculation Documentation

1.1 Introduction

The `energy_losses.py` script calculates transmission line losses and converter losses for a given project configuration. It exposes `get_total_energy_losses()` as the main entry point (used by `line_loss_costs.py` and `emissions.py`) and can be run standalone to print results. For reconductoring projects it uses the old configuration's voltage (towers unchanged) and the new configuration's resistance (new conductors).

1.2 Method Overview

Canonical notation. The canonical variable names and equations for energy losses are defined in `COST_METHODOLOGY.md` under “Energy/Emissions” / “Energy losses”. Key quantities: C_{new} , V , L , u , phase current I , full-load adjustment f_{load} , line loss $P_{\text{lineloss,MW}}$, converter loss $P_{\text{converterloss,MW}}$, annual and lifetime energy losses E_{loss} , $E_{\text{totalloss,lifetime}}$.

1.2.1 Purpose

This script calculates energy losses by:

1. Loading project technical and physical details
2. Building category identifier (`build_category_string`); for reconductoring, also building old category for voltage lookup
3. Loading circuit and resistance details for the (new and, if reconductoring, old) category
4. For reconductoring: using old voltage and new resistance; for greenfield: using new config throughout
5. Calculating phase current (`calculate_phase_current`), full load adjustment (`full_load_adjusted`)

6. Calculating line losses (`calculate_line_losses`) and converter losses (`calculate_converter_losses`, DC only)
7. Returning a dict of losses and inputs; standalone run displays results via `print_results`

1.2.2 Calculation Approach

The energy loss calculation follows this structure:

Line Losses:

$$I_{phase} = \frac{P}{\sqrt{3} \times V \times N_{circuits}} \quad (\text{AC}) \quad (1.1)$$

$$I_{phase} = \frac{P}{V \times N_{poles}} \quad (\text{DC}) \quad (1.2)$$

$$P_{loss} = I^2 \times R \times N_{phases} \times N_{circuits} \quad (1.3)$$

$$\text{Losses (MWh/year)} = P_{loss} \times \text{Full Load Adj} \times 8760 \quad (1.4)$$

Converter Losses (DC only):

$$P_{converter} = P \times \text{Utilization} \times \text{Loss Rate} \quad (1.5)$$

$$\text{Converter Losses (MWh/year)} = P_{converter} \times 8760 \quad (1.6)$$

1.2.3 Inputs and Outputs

Inputs:

- Project technical details (construction type, AC/DC, capacity, conductor type, converter type, line utilization, converter loss percentage; for reconductoring: old capacity, old conductor, old AC/DC)
- Physical details (line length via `load_physical_details`)
- Circuit and resistance details (via `load_circuit_and_resistance_details` for category; YAML keys: voltage, conductors per phase, phases, circuits/poles, AC_75_resistance, DC_20_resistance)

Outputs:

- Line losses (MW/mile, MWh/year, percentage)
- Converter losses (MW, MWh/year, percentage)
- Total losses (line + converter)
- Lifetime losses

1.2.4 Integration with Other Scripts

This script integrates with:

- `smart_loaders.py` — `load_project_technical_details`, `load_physical_details`, `load_circuit_and_resistance_details`, `get_project_data_raw`
- `calculation_utils.py` — `build_category_string`, `get_converter_type`, `calculate_phase_current`, `full_load_adjusted`, `calculate_line_losses`, `calculate_converter_losses`, `to_percent`
- `path_config.py` — `YAMLS_DIR` (for loader path context)
- `line_loss_costs.py` and `emissions.py` — import `get_total_energy_losses()` to obtain loss data

1.3 Key Functions

1.3.1 `get_total_energy_losses()`

Returns a dictionary of line and converter losses and inputs for the current project. For reconductoring, uses old configuration's voltage and new configuration's resistance.

Returns:

- `losses_mwh_per_year`, `lifetime_losses_mwh`, `losses_mw_per_mile`, `resistance_per_mile`, `line_loss_per_mile_percent`, `total_line_loss_mw`, `total_line_loss_percent`
- `total_converter_losses_mw`, `total_converter_losses_mwh`, `converter_loss_percent`
- `total_losses_mwh_per_year`, `project_lifetime`, `voltage_kv`, `phase_current`, `full_load_adj`, `line_length`, and other circuit parameters

Key Logic:

```
1 new_category = build_category_string(project_details=project_details
2 )
3 if project_details.reconductoring:
4     old_category = build_category_string(construction_type,
5     old_ac_dc, old_capacity, old_conductor, old_converter)
6     old_circuit = load_circuit_and_resistance_details(old_category)
7     voltage_kv = old_circuit.voltage_kv
8     new_circuit = load_circuit_and_resistance_details(new_category)
9     # use new_circuit for resistance and counts
10 else:
11     circuit = load_circuit_and_resistance_details(new_category)
12     voltage_kv = circuit.voltage_kv
13     # ...
14 phase_current = calculate_phase_current(capacity_mw, voltage_kv,
15     number_of_phases, number_of_circuits_poles, ac_dc)
```

```

14 full_load_adj = full_load_adjusted(project_details.line_utilization)
15 # calculate_line_losses(...), calculate_converter_losses(...,
    converter_loss_percentage=project_details.
    converter_loss_percentage)

```

1.3.2 print_results(...)

Prints organized results in sections (system configuration, line losses, converter losses, summary). Used by main() when run standalone.

1.3.3 main()

Calls get_total_energy_losses(), then print_results(...) with the returned dict and project details.

1.4 Example

The following example demonstrates a typical usage:

```

1 # Example: 500 MW Overhead AC line
2 # Category: "Overhead/AC/500MW/Advanced Aluminum/NA"
3 #
4 # Inputs (from YAML):
5 #   - Capacity: 500 MW
6 #   - Voltage: 345 kV
7 #   - Conductors per phase: 2
8 #   - Number of phases: 3
9 #   - Number of circuits: 1
10 #   - AC resistance (75 C): 0.05 ohms/mile
11 #   - Line utilization: 70%
12 #   - Line length: 100 miles
13 #   - Project lifetime: 50 years
14 #
15 # Calculation:
16 #   Phase current = 500 MW / (sqrt(3) * 345 kV * 1) = 837 A
17 #   Full load adj = 0.7^2 = 0.49
18 #   Line loss = (837^2 * 0.05 * 3 * 1) * 0.49 = 51.4 MW
19 #   Line loss % = 51.4 / 500 = 10.3%
20 #   Losses MWh/year = 51.4 * 8760 = 450,000 MWh/year
21 #   Lifetime losses = 450,000 * 50 = 22.5M MWh
22 #
23 # Output:
24 #   - Line loss: 0.514 MW/mile
25 #   - Line loss %: 10.3%
26 #   - Losses MWh/year: 450,000
27 #   - Lifetime losses: 22.5M MWh

```

Integration Example:

```
1 # Used by line_loss_costs.py and emissions.py:
2 from energy_losses import get_total_energy_losses
3
4 loss_data = get_total_energy_losses()
5 losses_mwh_per_year = loss_data["losses_mwh_per_year"]
6 lifetime_losses_mwh = loss_data["lifetime_losses_mwh"]
7 total_losses_mwh_per_year = loss_data["total_losses_mwh_per_year"]
8 # ... other keys as needed
```

1.5 Key Implementation Details

1.5.1 AC vs. DC Calculations

The script handles both AC and DC systems:

- **AC:** Uses $\sqrt{3}$ factor, resistance at 75°C
- **DC:** No $\sqrt{3}$ factor, resistance at 20°C

1.5.2 Full Load Adjustment

The full load adjustment accounts for the fact that losses are proportional to current squared, and average current is less than peak current due to utilization. The adjustment is typically u^2 where u is the utilization factor.

1.5.3 Converter Losses

Converter losses only apply to DC projects and are calculated in `calculation_utils.calculate_converter` based on number of converters, converter type, line utilization, capacity, and `converter_loss_percentage` from project details.

1.5.4 Resistance Values

The script uses different resistance values:

- **AC:** Resistance at 75°C (operating temperature)
- **DC:** Resistance at 20°C (reference temperature)

1.5.5 Multiple Circuits/Poles

The script accounts for multiple circuits (AC) or poles (DC) by multiplying losses by the number of circuits/poles.

1.5.6 Reconductoring: Old Voltage, New Resistance

For reconductoring projects, physical towers (and voltage clearances) are unchanged, so the script uses the *old* configuration's voltage for phase current. Conductor resistance comes from the *new* configuration (new conductors). Old category is built from `old_capacity_mw`, `old_conductor_type`, `old_ac_dc`, and `get_converter_type(old_ac_dc, ...)`.

1.5.7 Standalone vs. Library Use

This script can be run standalone (displays results via `print_results`) or imported for `get_total_energy_losses()`. The underlying calculation functions (phase current, full load adjustment, line losses, converter losses) live in `calculation_utils.py` for reuse.